Backbone: Developing Clinical Intelligence

Hiring Challenge / Contact: andrew@backbonesystems.ai

1. Overview

Backbone reviews clinical records to understand what care a patient received and whether the documentation supports the conclusions being drawn about that care. The information needed for a review is often spread across documents created by different people and systems.

You will receive separate documents from fictional inpatient behavioral health treatment episodes across multiple patients: clinical notes, attendance records, treatment plans, medication visits, authorizations, and administrative records. Build a working prototype that produces an accurate, auditable clinical abstraction and uses it to answer questions about individual patients and the collection as a whole.

Assume the production collection contains 500,000+ documents, with repeated reviews and new documents arriving daily. Implement your prototype on the smaller supplied datasets and demonstrate how it reuses work across questions and runs**, and can scale to much larger document sets.**

2. The Problem

Build a system that accepts the records in `documents/` and answers the five questions in `questions.json`. The questions cover:

- How many therapy sessions each patient attended, of which types, on how many days, and for how many hours overall and by week
- Whether the care delivered met the treatment plan's requirements in effect for each period
- Which patients had two consecutive weeks below those requirements, and which patients' inclusion depends on unresolved documentation
- How the types and amounts of care delivered changed before and after a treatment-plan change
- What the record says about progress for the patients specified in `questions.json`, and which observations or assessments support that account

Your implementation should retain its work so that it can answer new questions after restarting. New questions should use the existing abstraction; you may check relevant source passages, but should not rebuild the abstraction for each question. Duplicate copies of a document should not change clinical results. The source documents may describe the same contact more than once, disagree about details, or record a later correction. A scheduled appointment, a clinician's note, and an attendance entry can each contribute different information. A later document does not automatically override an earlier one.

Calculate numerical answers in code from the abstraction. A reviewer should be able to trace a finding across patients to the individual patients, services, calculations, and source passages that support it. Choose the implementation and storage you think best fit the problem.

Measure performance on the supplied documents. Report initial processing time, latency for individual-patient and collection-wide questions, model usage and cost where available, and the size of the saved abstraction. Explain where your implementation would become expensive or slow with 500K+ documents, which part of the code creates that constraint, and what you would change. Distinguish measured results from estimates

Backbone: Developing Clinical Intelligence

Hiring Challenge / Contact: andrew@backbonesystems.ai

3. Submission Criteria

Please email manan@backbonesystems.ai and andrew@backbonesystems.ai with a ZIP file or link to a Github repository containing:

1. Runnable code and setup instructions, including how to process documents and answer new questions using the saved abstraction.
2. Your clinical abstraction, answers to the five development questions, execution logs, and benchmark results, with supporting sources.
3. A short `README` explaining your approach, one design decision you tested and what you learned, an observed limitation, and how you would investigate it next. Identify the first bottleneck you expect at a million documents. Include model names and settings, any coding assistance used, and approximate runtime and model cost where available.

Spend 2-5 hours, and use any language, models, or libraries. The implementation should work on the supplied documents and related unseen questions without manually encoding patient facts or answers. Document incomplete work and tradeoffs.

4. Evaluation Criteria

Once you send a submission, we will schedule a follow-up 30-minute system design call. In this session, we will review your abstraction, follow a few conclusions back to their sources, and run related questions and additional documents through your submitted code. We will evaluate completeness, accuracy, reproducible calculations, evidence quality, handling of uncertainty, performance on follow-up questions, and research judgment.

5. FAQs

Can I use any programming language, model, or library?

Yes. Choose the tools you think best fit the problem, and include clear instructions for running your submission.

Do I need clinical knowledge or healthcare experience?

No. You should be able to derive the understanding you need from the supplied documents and, where helpful, online resources. We are evaluating your engineering and research judgment, not prior clinical knowledge. Patient-specific conclusions should be supported by the supplied records.

Should I approach this as a RAG exercise?

We recommend thinking beyond a standard vector-search-and-answer pipeline. Pay close attention to what the questions require: reconciling evidence across documents, calculating quantities over time, and distinguishing established conclusions from uncertainty. Retrieval may be useful, but consider what representation of the record would make these answers reliable, auditable, and straightforward to extend. We do not require a particular architecture.

What do you mean by a “clinical abstraction”?

An organized representation of the patient’s course of care that your system derives from the documents and uses to answer questions. Its format and level of detail are up to you. We should be able to inspect it, understand how it represents the relevant evidence, and trace its contents back to their sources.

What should I do when documents conflict or information is missing?

Make the disagreement or gap explicit. If you resolve a conflict, explain the basis for that decision and retain the supporting evidence. If the record does not support a definitive answer, explain what can be established and what additional documentation would be needed. State any assumptions that materially affect your answer.

What counts as sufficient supporting evidence?

References should be specific enough for a reviewer to locate the relevant evidence within a document. For calculated answers, show which records contributed to the result and how the calculation was performed, including consequential inclusion or exclusion decisions.

How broadly should the implementation generalize?

Focus on the supplied documents and the kinds of questions described above. The implementation should accommodate related unseen questions and additional documents without manually encoding this patient's facts or answers. You do not need to build a universal clinical review system.

How polished should the prototype be?

Keep the work within the 2-5-hour scope. A command-line interface or runnable notebook is sufficient; a UI is not required. Prioritize a working end-to-end implementation, inspectable outputs, and meaningful checks. Document any incomplete work, tradeoffs, and what you would investigate next.

Can I use AI coding assistants?

Yes. Please disclose the assistance used, along with model names and settings where applicable. Be prepared to explain your implementation, its design choices, and how you checked its outputs.